

# ■■ HIGH-LEVEL DESIGN SYSTEM DESIGN

16-Week Architecture & Scaling Masterclass · Interview Ready · Build Like Giants

by SYNTAX ERROR | [syntaxerrorr.com](https://syntaxerrorr.com) | [@code.abhii07](https://twitter.com/code.abhii07)

|       |        |       |          |
|-------|--------|-------|----------|
| 16    | 8      | 3 hrs | 100+     |
| Weeks | Phases | Daily | Concepts |

## ■ DAILY STUDY PLAN (3 hours)

|                    |                   |                |
|--------------------|-------------------|----------------|
| 1 hr               | 1 hr              | 1 hr           |
| ■ Concept Learning | ⇒ ■ Draw Diagrams | ■ Case Studies |

## ■ GOLDEN RULES — THINK IN SYSTEMS

|   |                                                                          |
|---|--------------------------------------------------------------------------|
| ■ | Don't just read theory — understand the WHY behind every design decision |
| ■ | Don't memorize solutions — think in trade-offs and real-world scenarios  |
| ■ | Draw system diagrams for every concept — visualize everything you learn  |
| ■ | Practice case studies daily — be interview-ready, design at scale        |

## ■ 8-PHASE ROADMAP

|        |                 |                 |             |               |             |          |              |
|--------|-----------------|-----------------|-------------|---------------|-------------|----------|--------------|
| P1     | P2              | P3              | P4          | P5            | P6          | P7       | P8           |
| Basics | Core Components | Database Design | Scalability | Communication | Reliability | Advanced | Case Studies |

## ■ ■ PHASE 1 — Basics (Week 1–2)

### Phase 1    **BASICS** · Week 1–2 · 2 Weeks

■ What is High-Level Design & System Design — the big picture ■ Client–Server Architecture fundamentals — request/response cycle ■ Monolithic vs Microservices Architecture — trade-offs, when to use which ■ REST APIs — HTTP methods, status codes, statelessness ■ Key Metrics — Latency, Throughput, Scalability, Availability, Reliability

## ■ ■ PHASE 2 — Core Components (Week 3–5)

### Phase 2    **CORE COMPONENTS** · Week 3–5 · 3 Weeks

■ Load Balancer — Round Robin, Least Connections, Sticky Sessions ■ Caching Strategies — Redis basics, Cache invalidation (LRU, TTL) ■ CDN (Content Delivery Network) — Global content distribution ■ Reverse Proxy — Request routing, SSL termination, security benefits ■ API Gateway — Rate limiting, authentication, routing, request aggregation

## ■ ■ PHASE 3 — Database Design & Scaling (Week 6–7)

### Phase 3    **DATABASE DESIGN** · Week 6–7 · 2 Weeks

■ SQL vs NoSQL — when to use which, pros and cons of each ■ Indexing Strategies — B-tree, Hash indexing, composite indexes ■ Query Optimization — EXPLAIN plans, slow query logs, N+1 problem ■ Scaling Databases — Sharding (range, hash, directory-based) ■ Replication — Master-Slave, Master-Master, eventual consistency ■ CAP Theorem — Consistency, Availability, Partition Tolerance

## ■ ■ PHASE 4 — Scalability (Week 8–9)

### Phase 4 SCALABILITY · Week 8–9 · 2 Weeks

■ Vertical Scaling — Adding more resources to a single server (scale up) ■ Horizontal Scaling — Distributing load across multiple servers (scale out) ■ Auto-scaling — Dynamic capacity management based on traffic ■ Stateless vs Stateful Systems — implications for horizontal scaling ■ Session Management — Sticky sessions, distributed sessions, JWT ■ Rate Limiting — Token bucket, leaky bucket algorithms

## ■ ■ PHASE 5 — Communication Patterns (Week 10)

### Phase 5 COMMUNICATION · Week 10 · 1 Week

■ Synchronous vs Asynchronous Communication patterns — trade-offs ■ Message Queues — Kafka, RabbitMQ, AWS SQS — use cases and differences ■ Pub-Sub Messaging — Decoupling services, fan-out pattern ■ Dead Letter Queues — Handling failed messages, retry strategies

## ■ ■ PHASE 6 — Reliability & Security (Week 11)

### Phase 6 RELIABILITY & SECURITY · Week 11 · 1 Week

■ Fault Tolerance — Building resilient systems that survive failures ■ Failover Mechanisms — Active-Passive, Active-Active strategies ■ Data Backup & Recovery — WAL (Write-Ahead Logging), snapshots ■ Health Checks — Heartbeat monitoring, liveness & readiness probes ■ Authentication & Authorization — OAuth2, JWT, RBAC

## ■ ■ PHASE 7 — Advanced Concepts (Week 12–13)

### Phase 7    **ADVANCED CONCEPTS** · Week 12–13 · 2 Weeks

■ Consistent Hashing — Reducing cache misses in distributed systems ■ Distributed Systems Basics — Consensus algorithms, ACID, BASE ■ Event-Driven Architecture — Event sourcing, CQRS pattern ■ Service Mesh — Istio, traffic management, service discovery ■ Circuit Breaker Pattern — Preventing cascading failures

## ■ ■ PHASE 8 — Interview-Ready Case Studies (Week 14–16)

### Phase 8    **SYSTEM DESIGN CASE STUDIES** · Week 14–16 · 3 Weeks

- |   |                                                                                                                |
|---|----------------------------------------------------------------------------------------------------------------|
| 1 | <b>URL Shortener</b> Like Bitly — High availability, caching, analytics, hash collision handling               |
| 2 | <b>Instagram / Facebook Feed</b> Billions of users — Feed generation, ranking algorithms, push vs pull         |
| 3 | <b>WhatsApp Chat System</b> Real-time messaging — WebSockets, message ordering, delivery receipts              |
| 4 | <b>YouTube / Netflix</b> Video streaming — CDN, encoding pipeline, recommendations, DRM                        |
| 5 | <b>Uber / Ride-Sharing</b> Real-time matching — Geospatial indexing, driver-rider communication, surge pricing |

## ■ ■ TOOLS & TECHNOLOGIES — Your System Design Arsenal

### ■ CACHING & DATABASES

|            |                              |                                                  |
|------------|------------------------------|--------------------------------------------------|
| Redis      | — In-memory data store       | Caching, sessions, leaderboards, pub-sub         |
| Memcached  | — Distributed memory caching | Session storage, simple key-value cache          |
| MongoDB    | — NoSQL document DB          | Flexible schema, horizontal scaling, geo queries |
| PostgreSQL | — Relational DB              | ACID compliance, advanced indexing, JSON support |

### ■ MESSAGE QUEUES & STREAMING

|              |                               |                                                  |
|--------------|-------------------------------|--------------------------------------------------|
| Apache Kafka | — Distributed event streaming | High throughput, fault tolerance, log compaction |
| RabbitMQ     | — Message broker              | Reliable message delivery, flexible routing      |
| AWS SQS      | — Managed queue service       | Serverless, simple API, at-least-once delivery   |

### ■ ■ INFRASTRUCTURE & DEPLOYMENT

|                   |                    |                                                 |
|-------------------|--------------------|-------------------------------------------------|
| Docker            | — Containerization | Consistent environments across dev/staging/prod |
| Kubernetes        | — Orchestration    | Auto-scaling, self-healing, rolling updates     |
| AWS / GCP / Azure | — Cloud Platforms  | EC2, Load Balancers, CDN, managed databases     |

## ■ RESOURCES & LEARNING

|                            |                                                                           |                                                                          |
|----------------------------|---------------------------------------------------------------------------|--------------------------------------------------------------------------|
| ■ System Design Primer     | <a href="https://github.com/donnemartin">github.com/donnemartin</a>       | Comprehensive open-source system design guide — #1 resource on GitHub    |
| ■ Syntax Error             | <a href="https://syntaxerrorr.com">syntaxerrorr.com</a>                   | HLD/System Design notes, videos, and in-depth explanations — our website |
| ■ ■ System Design Playlist | <a href="https://youtube.com/@code.abhii07">youtube.com/@code.abhii07</a> | Complete video course on System Design — interview prep from scratch     |

## ■ PHASE MILESTONE TRACKER

| Phase     | Key Deliverables                                      |
|-----------|-------------------------------------------------------|
| Phase 1–2 | Design Client-Server, Load Balancer, Caching diagrams |
| Phase 3   | Database schema design, Sharding strategy docs        |
| Phase 4   | Scalability plan — horizontal + auto-scaling design   |

|           |                                                         |
|-----------|---------------------------------------------------------|
| Phase 5–6 | Message queue architecture, Failover system design      |
| Phase 7   | Consistent hashing + event-driven architecture diagram  |
| Phase 8   | 5 full case study write-ups — interview ready portfolio |

|              |                                                 |                                                |
|--------------|-------------------------------------------------|------------------------------------------------|
| SYNTAX ERROR | Our Website<br><a href="#">syntaxerrorr.com</a> | Our Instagram<br><a href="#">@code.abhii07</a> |
|--------------|-------------------------------------------------|------------------------------------------------|

*"Great architects don't memorize solutions — they understand trade-offs. Design with scalability in mind, not for the problem of today. ■■"*